

BASTION LINE

Rapport Technique Officiel — V2

Emprise chemin/construction, sélecteur tactile, pilote asset Leonardo

Date du rapport	26 septembre 2026
Dépôt	github.com/scuillerjack-oss/bastion-line-v0
Commit de départ (V1 vérifiée)	5f2e1403d828a23aeaa2001d9e8ba30f2aa00905
Commit final V2 (HEAD distant vérifié)	9aa3a0b746771d84bf111c221f5fa024c1abd25f
Identifiant de build vérifié	v0.1.0+9aa3a0b (rebuild local GITHUB_SHA=HEAD, confirmé identique)
URL publique	https://scuillerjack-oss.github.io/bastion-line-v0/
Statut	Candidate à la bêta physique V2 — NON VALIDÉE, décision humaine en attente

1. État Git initial constaté

- Workspace propre (`git status --short` vide) avant toute modification.
 - HEAD local = 5f2e1403d828a23aeaa2001d9e8ba30f2aa00905, branche main.
 - HEAD distant réel (`git ls-remote`) : identique — aucune divergence.
 - Ce commit correspond exactement à la référence V1 citée par le cahier des charges V2 (section 0). La PWA V1 a été installée et testée avec succès par le porteur sur son téléphone physique : aucun chantier PWA n'a été rouvert dans cette mission, conformément à l'exigence du cahier.
-

2. Priorité 1 — Emprise visuelle construction/chemin

2.1 Diagnostic de la cause systémique

Aucune vérification n'a jamais existé, à aucune version antérieure, entre la position d'un emplacement de construction et la géométrie réelle du chemin. L'audit programmatique de tous les emplacements des 5 niveaux (distance point-segment réelle vers chaque segment de chaque chemin) a révélé :

Niveau / emplacement	Distance au chemin	Constat
Niveau 2, "s2" (130,250)	0	Placé EXACTEMENT sur la ligne centrale du chemin — le "canon posé sur la route" rapporté en bêta
Niveau 5, "s5" (200,420)	0	Placé EXACTEMENT sur la ligne centrale du chemin commun après convergence des deux voies
Niveau 1, "s3" (40,300)	50	Sous la marge de sécurité réaliste une fois les anneaux de palier et un futur sprite pris en compte

Les 36 autres emplacements réels des 5 niveaux respectaient déjà une marge correcte.

2.2 Correction structurelle (pas un correctif ponctuel)

- Nouveau `src/engine/footprint.js` : distance point-segment réelle, partagée par le rendu et par la validation de niveau — une seule source de vérité.
 - `PATH_WIDTH` déplacé depuis une constante dupliquée dans `render.js` vers `engine/constants.js` (root cause de la divergence possible entre rendu et validation).
 - Nouvelle constante `TOWER_FOOTPRINT_RADIUS=38` : couvre à la fois les anneaux de palier des tours Canvas existantes (jusqu'à -36px au palier maximum) ET le sprite Leonardo, délibérément dimensionné pour rester dans cette même enveloppe (voir section 4).
 - `validateLevel()` alerte désormais sur toute violation d'emprise — garde-fou PERMANENT, vérifié par un test sur les 5 niveaux réels à chaque exécution de la suite.
 - Les 3 emplacements en violation ont été repositionnés (déplacements minimaux, aucun autre emplacement touché) ; aucune trajectoire, collision, portée ou logique de ciblage modifiée.
-

3. Priorité 2 — Sélecteur de construction proche des bords

3.1 Diagnostic de la cause réelle (pas une supposition)

Le panneau de construction est une feuille ancrée en bas de l'écran (`.panel-root { position: absolute; bottom: 0; }`). Ouvrir ce panneau insère un vrai `<button>` DANS LE DOM de façon synchrone, à l'intérieur même du gestionnaire `pointerdown` du canvas — avant que le doigt qui vient de tapoter ne se soit relâché. Quand l'emplacement tapé est proche du bas, ce nouveau bouton apparaît

exactement sous le doigt encore posé. Au relâchement, le navigateur synthétise un événement `click` de compatibilité en fonction de l'élément RÉELLEMENT présent à cet endroit à ce moment — ce nouveau bouton, pas le canvas — et déclenche donc une construction immédiate depuis un seul appui. C'est un comportement standard et documenté de la spécification Pointer Events, pas un bug de rendu.

3.2 Correction à la cause (pas un délai arbitraire)

Annuler l'événement `pointerdown` d'origine (`ev.preventDefault()`) supprime, par spécification, la génération du `click` de compatibilité pour la suite de ce même pointeur — quel que soit l'élément qui se retrouve sous le doigt ensuite. Ajouté dans `src/ui/input.js`, sur le seul gestionnaire `pointerdown` du canvas (jamais sur les boutons du panneau eux-mêmes, qui continuent de répondre normalement à un second appui volontaire).

3.3 Positionnement adaptatif

Le sélecteur s'ancre désormais en HAUT de l'écran quand la position ÉCRAN réelle (calculée via le même viewport que la couche tactile, pas seulement la coordonnée arène) de l'emplacement/tour tapé dépasse 60% de la hauteur de la fenêtre — sinon il reste ancré en bas comme en V1. Les emplacements proches du haut ne changent jamais de comportement.

4. Priorité 3 — Pilote d'intégration Leonardo (tour rapide/arbalète)

4.1 Fichier source et traitements appliqués

Fichier original conservé intact : `assets/leonardo/tour_rapide_arbalette_original.jpg` (1220x2712, jamais modifié). Traitements appliqués par `scripts/process_leonardo_asset.py` (script documenté, reproductible) :

- Retrait du bandeau noir (letterbox) haut/bas de l'export original, détecté par luminosité moyenne des lignes.
- Recadrage sur la boîte englobante réelle de la tour (fond gris exclu), avec une marge de 4%.
- Suppression du fond gris uniforme (172,172,172) par une carte alpha à TRANSITION DOUCE (jamais un seuil binaire, qui aurait produit un contour en dents de scie une fois réduit à la taille d'affichage réelle).
- Redimensionnement à 360x504 px (rééchantillonnage Lanczos) — résolution volontairement plus grande que la taille d'affichage en jeu (34x48 px logiques) pour rester net jusqu'à `devicePixelRatio=2`, sans variantes de résolution supplémentaires (non nécessaires à cette échelle).
- Compression PNG (`optimize=True`) : fichier final 252 Ko.

Aucun redessin, aucune réinterprétation, aucune fabrication des 3 autres tours à partir de cet asset.

4.2 Intégration technique minimale et réutilisable

- Nouveau `src/ui/sprites.js` : chargement d'image minimal (pas un moteur de sprites complet), état observable ("`loading`" / "`loaded`" / "`error`") — réutilisable pour de futurs assets.
- `render.js` : si l'asset est chargé, il remplace entièrement la silhouette Canvas de la tour "rapide" ; sinon (chargement en cours ou échoué), fallback immédiat et silencieux sur la silhouette Canvas V1 — jamais d'écran cassé ni de tour invisible.
- Dimensions d'affichage (34x48, ancrage bas +14) choisies pour rester DANS l'enveloppe d'emprise partagée avec les 3 autres tours (section 2) — l'intégration respecte donc nativement la correction d'emprise chemin/construction, sans règle spéciale.



Tour Leonardo affichée à la taille de jeu réelle (deviceScaleFactor=3, capture automatisée), niveau 1.

Rappel du cahier : cette capture documente que l'asset se charge et s'affiche réellement en jeu, à sa taille réelle, sans chevaucher le chemin. Elle ne constitue PAS une validation esthétique — celle-ci revient exclusivement à la bêta physique du porteur.

5. Fichiers modifiés / ajoutés

Fichier	Nature
src/engine/footprint.js	Ajouté — validation géométrique emprise/chemin
src/engine/constants.js	Modifié — PATH_WIDTH, TOWER_FOOTPRINT_RADIUS, FOOTPRINT_SAFETY_MARGIN
src/engine/levels.js	Modifié — 3 emplacements repositionnés, garde-fou branché dans validateLevel
src/ui/input.js	Modifié — preventDefault() sur pointerdown (cause du clic fantôme)
src/main.js	Modifié — positionnement adaptatif du panneau (haut/bas)
src/style.css	Modifié — variante .panel-root--top
src/ui/render.js	Modifié — intégration du sprite Leonardo pour la tour rapide
src/ui/sprites.js	Ajouté — chargement d'asset minimal et réutilisable
assets/leonardo/tour_rapide_arbalette_original.jpg	Ajouté — fichier source ORIGINAL, intact
public/assets/towers/tour_rapide_arbalette.png	Ajouté — asset traité (voir section 4.1)
scripts/process_leonardo_asset.py	Ajouté — script de traitement documenté et reproductible
tests/footprint.test.js	Ajouté — 11 tests
tests/sprites.test.js	Ajouté — 2 tests
scripts/check-mobile.mjs	Modifié — 4 nouveaux tests Playwright

6. Tests et résultats complets

Suite	Résultat
Tests unitaires (node:test)	66/66 passants (53 hérités de V1 + 13 nouveaux)
Vérifications mobiles/PWA (Playwright)	22/22 passantes (18 héritées de V1 + 4 nouvelles)

Aucune assertion existante n'a été supprimée ni affaiblie. Les 4 nouveaux tests mobiles couvrent : un appui unique sur un emplacement réel proche du bas (niveau 1, "s5") n'ouvre que le sélecteur, ancré en haut, sans construction ; une seconde action volontaire construit bien la tour choisie ; un emplacement proche du haut garde le comportement V1 ; l'asset Leonardo charge réellement en HTTP 200 dans le build de production ET le jeu reste jouable sans erreur si son chargement est délibérément bloqué (fallback vérifié, pas supposé).

7. Git, CI/déploiement — vérification réelle

Étape	Résultat vérifié
Commit final	9aa3a0b746771d84bf111c221f5fa024c1abd25f
HEAD distant après push	Vérifié via git ls-remote — identique au commit local
Job CI "build"	success (66 tests + 22 vérifications mobiles exécutés en CI)
Job CI "deploy"	success — log lu directement, pages_build_version = 9aa3a0b746771d84bf111c221f5fa024c1abd25f
Build ID re-vérifié localement	v0.1.0+9aa3a0b (rebuild avec GITHUB_SHA=HEAD réel)

Asset Leonardo dans le build servi	Présent (dist/assets/towers/tour_rapide_arbalete.png), vérifié
URL publique	https://scuillerjack-oss.github.io/bastion-line-v0/

8. Limites restant à valider sur téléphone physique

- **Validation esthétique de l'asset Leonardo** : ce rapport documente que l'asset charge, s'affiche à la bonne taille et ne chevauche pas le chemin — il ne se prononce PAS sur si le résultat est "beau" ou cohérent avec les 3 autres tours restées en Canvas. C'est explicitement la décision du porteur.
 - **Marge d'emprise en conditions réelles** : la marge de 38px de rayon d'emprise a été calculée pour couvrir le sprite Leonardo ET les anneaux de palier existants avec un buffer de sécurité, mais seule une bêta physique confirmera qu'aucun chevauchement résiduel n'est perceptible sur un écran réel.
 - **Sélecteur tactile** : corrigé et vérifié par un vrai tap tactile en environnement automatisé (Playwright + deviceScaleFactor=3) sur l'emplacement réel du niveau 1 le plus proche du bas — reste à confirmer sur le téléphone physique du porteur, y compris sur d'autres tailles d'écran.
 - **Performance réelle** : le sprite Leonardo ajoute un drawImage par tour "rapide" posée, remplaçant quelques primitives Canvas — coût attendu négligeable, non mesuré sur l'appareil physique du porteur dans cette session.
-

9. Conclusion

Les trois priorités du cahier des charges V2 ont été traitées comme une seule mission cohérente, chacune corrigée à sa cause réelle plutôt que par un correctif local : une validation géométrique permanente pour l'emprise chemin/construction, une correction fondée sur la spécification Pointer Events pour le sélecteur tactile, et une intégration minimale et réutilisable pour le pilote graphique Leonardo. Le moteur, les vagues, l'économie, les dégâts, la progression et la PWA V1 restent inchangés.

Conformément à l'instruction explicite du cahier, BASTION LINE V2 n'est PAS déclarée validée par ce rapport. Ce livrable est une candidate à la bêta physique V2. La décision sur le ressenti de jeu et sur la validité esthétique de l'asset Leonardo appartient exclusivement au porteur, après son propre test sur téléphone physique.